

TP MATH INFORMATIQUE

Tonin PACHER

March 16, 2026

1 Matrice2D

toutes les matrices aléatoire seront de taille (8,16)
on peut donc déjà définir les bornes max pour les différentes entropies :

$$\max(H(X)) = \log 2(8) = 3$$

$$\max(H(Y)) = \log 2(16) = 4$$

$$\max(H(X, Y)) = \log 2(128) = 7$$

$$\max(I(X; Y)) = \min(H(X), H(Y))$$

1.1 Codage de l'Entropie H(X)

Première étape nous allons coder l'entropie classique se basant sur une seule variable aléatoire ici H(X)

rapellons la formule :

$$\sum_{n=1}^N p(xi) \times \log 2(p(xi))$$

le code python est donc :

```
1 def Entropie(Z):
2     """ entropie de H(X) """
3     Z = Z + eps
4     Z = Z / Z.sum()
5     Px = np.sum(Z, axis=1)
6     return -np.sum(Px * np.log2(Px))
```

apres execution sur une matrice 2D cela nous donne :

```
[[459 526 195 684 501 754 763 479 319 798 850 525 874 363 785 319]
 [321 261 491 466 320 8 762 932 49 73 964 111 565 87 822 347] [584 885
 472 564 862 425 822 368 519 823 522 609 910 161 255 507] [137 825 636
 726 215 927 103 572 95 27 193 173 573 840 792 880] [992 585 581 573 417
```

*143 539 221 608 309 754 724 43 953 608 108] [517 381 899 727 261 823
518 418 195 272 1 65 442 988 18 589] [140 914 927 958 35 311 438 35 180
494 568 298 594 330 197 179] [40 960 156 417 6 818 6 611 77 231 947 239
899 935 109 513]]*

nous donne : 2.9875584206597683

**la borne max noté plus haut est 3 c'est donc coherent etant donné
qu'on est sur un tirage aléatoire l'incertitude et très élevé
après teste sur un dirac on a bien 0**

1.2 Codage de l'Entropie jointe $H(X,Y)$

deuxième etape nous allons coder l'entropie jointe, on rapelle la formule :

$$\sum_{n=1}^N p(xi, yj) \times \log 2(p(xi, yj))$$

le code python est donc :

```
1 def Entropie_jointe(Z):
2     """ entropie jointe entre H(X) et H(Y) """
3     Z = Z + eps
4     P_Z = Z / np.sum(np.ndarray.flatten(Z))
5     P_flat = P_Z.flatten()
6     return -1 * np.sum((P_flat) * np.log2(P_flat))
```

apres execution sur une matrice 2D cela nous donne :

*[[[981 817 354 961 359 887 905 787 486 567 940 269 150 824 465 2] [4
41 242 261 204 554 413 635 14 224 127 553 550 915 683 878] [559 72 497
686 778 31 534 309 591 78 386 561 118 523 150 702] [274 597 849 847 827
935 441 635 307 470 72 398 352 400 186 304] [65 495 78 15 563 768 803
729 318 909 876 753 668 525 428 471] [530 510 125 277 322 405 244 221
436 345 577 998 756 623 343 453] [126 632 693 653 637 440 197 101 456
376 229 398 319 692 222 620] [620 786 701 916 901 574 580 972 96 335
463 265 390 831 682 924]]*

nous donne : 6.746527033542511

**c'est coherent car la borne max noté plus haut est 7 et comme
on fait un tirage aléatoire on se rapproche de la borne max car
l'incertitude est élevé**

1.3 Codage de l'Entropie conditionnel $H(X|Y)$

Pour coder l'entropie conditionnel nous allons utiliser les formules ecrite au
dessus, on rapelle la formule de l'entropie conditionnel :

$$H(X, Y) - H(Y)$$

le code python est donc :

```
1 def Entropie_conditionnel(Z):
2     """ calcul de l'entropie conditionnel H(X | Y) """
3     J = Entropie_jointe(Z)
4     E = Entropie_Y(Z)
5     return J - E
```

il faut prendre en compte que la fonction Entropie Y existe, il faut juste changer la valeur de axis dans l'entropie de X

apres execution sur une matrice 2D cela nous donne :

```
[[981 817 354 961 359 887 905 787 486 567 940 269 150 824 465 2] [ 4
41 242 261 204 554 413 635 14 224 127 553 550 915 683 878] [559 72 497
686 778 31 534 309 591 78 386 561 118 523 150 702] [274 597 849 847 827
935 441 635 307 470 72 398 352 400 186 304] [ 65 495 78 15 563 768 803
729 318 909 876 753 668 525 428 471] [530 510 125 277 322 405 244 221
436 345 577 998 756 623 343 453] [126 632 693 653 637 440 197 101 456
376 229 398 319 692 222 620] [620 786 701 916 901 574 580 972 96 335
463 265 390 831 682 924]]
```

nous donne : **3.7670756143714543**

1.4 Codage de l'information mutuelle $I(X;Y)$

pour coder l'information mutuelle de X et Y nous allons utiliser les formules rédigé à l'avance pour des questions de facilité, on rapelle que la formule est donc :

$$H(X) + H(Y) - H(X, Y)$$

le code python est donc :

```
1 def Information_Mutuelle(Z):
2     """ calcul de l'information mutuelle I(X;Y) """
3     J = Entropie_jointe(Z)
4     Ex = Entropie(Z)
5     Ey = Entropie_Y(Z)
6     return Ex + Ey - J
```

apres execution sur une matrice 2D cela nous donne :

[[981 817 354 961 359 887 905 787 486 567 940 269 150 824 465 2] [4
41 242 261 204 554 413 635 14 224 127 553 550 915 683 878] [559 72 497
686 778 31 534 309 591 78 386 561 118 523 150 702] [274 597 849 847 827
935 441 635 307 470 72 398 352 400 186 304] [65 495 78 15 563 768 803
729 318 909 876 753 668 525 428 471] [530 510 125 277 322 405 244 221
436 345 577 998 756 623 343 453] [126 632 693 653 637 440 197 101 456
376 229 398 319 692 222 620] [620 786 701 916 901 574 580 972 96 335
463 265 390 831 682 924]]

nous donne :0.21092405738769138

on sait que la borne max de l'information mutuelle est le plus petit entre $H(X)$ et $H(Y)$ apres calcul on remarque que la valeur est tres basse on peut donc en conclure que les informations X et Y n'ont pas beaucoup de lien

2 Matrice3D

Nous serons tout le long sur une matrice 3D de taille (8,16,7)
on peut donc définir les bornes max :

$$\max(H(A)) = \log 2(8) = 3$$

$$\max(H(B)) = \log 2(16) = 4$$

$$\max(H(C)) = \log 2(7) = 2.807$$

$$\max(H(X, Y)) = \log 2(128) = 7$$

$$\max(H(A, B, C)) = \log 2(896) = 9.8$$

$$\max(H(A, B|C)) = \log 2(128) = 7$$

2.1 Codage de l'entropie de la derniere variable 3D

pour calculer l'entropie de la 3D il faut faire la somme de l'axis 1 puis avec ces resultats on re somme sur axis 0 afin d'obtenir le total de la 3D, attention a bien renormaliser apres calcul

$$H = - \sum_{c=0}^{C-1} \left(\frac{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} (Z_1[a, b, c] + \varepsilon)}{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \sum_{c=0}^{C-1} (Z_1[a, b, c] + \varepsilon)} \right) \log_2 \left(\frac{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} (Z_1[a, b, c] + \varepsilon)}{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \sum_{c=0}^{C-1} (Z_1[a, b, c] + \varepsilon)} \right)$$

la fonction est un peu differente de l'entropie 2D car on compresse les axis qu'on ne veut pas, étant donné qu'ici on est en 3d il y a deux coté à supprimer, axis 0 et axis 1

le code python est donc :

```

1 def Entropie3D(Z1):
2     """ entropie de H(C) en 3 eme dimension """
3     Z1 = Z1 + eps
4     Z1 = Z1 / Z1.sum()
5     PC = np.sum(Z1, axis = (0,1))
6     PC = PC / np.sum(PC) #normalisation de nouveau
7     return -np.sum(PC * np.log2(PC))

```

Je ne mettrai pas les exemples car les matrices sont trop grande cela prendrai trop de place, le résultat sorti sur une matrice généré aléatoirement est ;

2.806586742493664

on remarque qu'on est très proche de la borne max on a donc une incertitude qui est maximale

2.2 Codage de l'entropie conditionnel jointe de $H(A, B | C)$

Pour calculer cela on va utiliser des fonctions ecrites avant, la formule est :

$$H(A, B | C) = H(A, B, C) - H(C)$$

le code python est donc :

```

1 def Entropie_jointeC3D(Z1):
2     Hc = Entropie3D(Z1)
3     Habc = Entropie_jointe3D
4     return Habc - Hc

```

on applique simplement la fomrulle avec les fonctions codé auparavant et on obtient : **6.7098072396548885**

on a donc un resultat tres proche du max on peut donc en deduire que A et B depende très peu de C

2.3 Codage de l'information mutuelle $I(A,B;C)$

pour cela on aura besoin de la fonction $H(A,B)$:

```

1 def Entropie_jointe2D(Z1, axis):
2     """ Entropie jointe de deux variables
3     en marginalisant la troisieme """
4     Z1 = Z1 + eps
5     Z1 = Z1 / Z1.sum()
6     P_2D = np.sum(Z1, axis=axis)
7     P_2D = P_2D / np.sum(P_2D)
8     P_flat = P_2D.flatten()
9     return -np.sum(P_flat * np.log2(P_flat))

```

ensuite on applique la formule :

$$H(A, B) + H(C) - H(A, B | C)$$

le code python est donc :

```

1 def Information_mutuelle3D(Z1):
2     """I(A,B;C) = H(A,B) + H(C) - H(A,B,C) """
3     H_AB = Entropie_jointe2D(Z1, axis=2) # marginalise C
4     H_C   = Entropie3D(Z1)               # H(C)
5     H_ABC = Entropie_jointe3D(Z1)        # H(A,B,C)
6     return H_AB + H_C - H_ABC

```

on obtient : **0.24095957591253025**

3 Hypercube

3.1 Codage de l'information mutuelle dirigé

La fonction `info_dirigee` calcule la somme des informations mutuelles conditionnelles de la forme :

$$ID(W) = \sum_{t=1}^{N-1} I(X_t; X_{t+1} | X_1, \dots, X_{t-1})$$

où $I(A; B | C)$ représente l'information mutuelle conditionnelle entre A et B sachant C , et N est le nombre de pas temporels (ici $N = 8$).

3.2 Codage de l'information dirigée sur la matrice 3D

où $I(A; B | C)$ est l'information mutuelle conditionnelle, et N est le nombre de pas temporels de la matrice.

le code python est donc :

```

1 def Information_Mutuelle_Conditionnelle(X, Y, Z):
2     H_XZ = Entropie_jointe(np.concatenate([X, Z], axis=1))
3     H_YZ = Entropie_jointe(np.concatenate([Y, Z], axis=1))
4     H_Z   = Entropie_jointe(Z)
5     H_XYZ = Entropie_jointe3D(np.concatenate([X, Y, Z], axis=1))
6     return H_XZ + H_YZ - H_Z - H_XYZ
7
8 def info_dirigee(W):
9     N = W.shape[1]
10    somme = 0
11    for t in range(1, N):
12        A = W[:, t, :][:, np.newaxis, :]
13        B = W[:, :t, :]
14        C = W[:, :t+1, :]
15        somme += Information_Mutuelle_Conditionnelle(C, A, B)
16    return somme

```

Après execution sur une matrice 3D générée aléatoirement :

```

1 W = np.random.random((8,8,20))
2 resultat = info_dirigee(W)
3 print("R sultat info dirigee :", resultat)

```

Résultat obtenu : 1.64

Analyse : La valeur positive indique que les vecteurs successifs de la matrice partagent une certaine dépendance temporelle. Plus la valeur est élevée, plus l'information est dirigée entre les étapes temporelles.

3.3 Estimation de la borne maximale via l'information mutuelle des derniers vecteurs

Pour obtenir une borne supérieure théorique de l'information dirigée, on peut calculer l'information mutuelle entre les dernières tranches de la matrice 3D :

$$I(X_N; Y_N) = H(X_N) + H(Y_N) - H(X_N, Y_N)$$

le code python est donc :

```

1 def I_XN_YN_3D(W):
2     XN = W[:, -1, :]
3     YN = W[:, -1, :]
4     XN = XN / XN.sum() + eps
5     YN = YN / YN.sum() + eps
6     H_X = -np.sum(XN * np.log2(XN))
7     H_Y = -np.sum(YN * np.log2(YN))
8     XY = np.concatenate([XN, YN], axis=1)
9     XY = XY / XY.sum() + eps
10    H_XY = -np.sum(XY * np.log2(XY))
11    return H_X + H_Y - H_XY
12
13 res = I_XN_YN_3D(W)
14 print("la borne max est ", res)

```

Résultat obtenu : 6.06 la valeur calculée au dessus est : 1.64 donc on est assez loin de la borne max

Analyse : Cette borne maximale représente la quantité maximale d'information que la dernière tranche de la matrice peut contenir sur la précédente. Comparée au résultat de l'information dirigée, elle permet d'évaluer l'efficacité de la transmission d'information dans la matrice.

4 Étude de données réelles sur des cachalots

Cette partie vise à identifier l'individu "Alpha" au sein d'un groupe de 6 cachalots (Cesar, Fanny, Marius, Norine, Felix, Honore) en utilisant l'Information Dirigée sur des données de trajectoires réelles.

Attention dans la quantification il y a un lien entre les valeurs numériques et les probas des valeurs de codage

4.1 récupération des données

dans un premier temps nous allons récupérer les données des fichiers csv et stocker les données qui nous interesse dans un dictionnaire que l'on appellera DATA

```
1 DOSSIER = os.path.dirname(os.path.abspath(__file__))
2
3 fichiers = {
4     "Cesar": "Trajectoire_Cesar_sec.csv",
5     "Fanny": "Trajectoire_Fanny_sec.csv",
6     "Marius": "Trajectoire_Marius_sec.csv",
7     "Norine": "Trajectoire_Norine_sec.csv",
8     "Felix": "Trajectoire_Felix_sec.csv",
9     "Honore": "Trajectoire_Honore_sec.csv"
10 }
11 #recuperations des fichiers csv dans un dictionnaire
12 data = {} #dictionnaire dans dictionnaire
13 for nom, f in fichiers.items():
14     chemin = os.path.join(DOSSIER, f) #construit le chemin vers les fichiers
15     try:
16         df = pd.read_csv(chemin, header=None) #lecture des fichiers csv
17         data[nom] = { #permet de stocker seulement les clonnes qui nous inte
18             'long': df[1].values, #definition longitude latitude la colon
19             'lat': df[2].values,
20             'prof': df[3].values,
21             'sec': df[5].values
22         }
23         print(f"OK : {nom}") #confirmation de la lecture les fichiers
24     except Exception as e:
25         print(f"Fichier manquant : {chemin} ({e})") #gestions des cas d'erre
26
27 noms = list(data.keys()) #creation d'une liste de toutes les cle (nom des an
28 N = len(noms) #nombre d'animaux r cup r s
29 print(f"\n{N} animaux charg s : {noms}\n")
30
31 if N < 2:
32     raise RuntimeError("Pas assez de fichiers charg s.") #en cas de probl m
```

4.2 mise en place de la synchronisation

Dans cette partie, nous allons synchroniser les données. Le problème majeur est que les instants d'enregistrement ne sont pas alignés entre les animaux :

chaque balise GPS émet à ses propres instants, indépendamment des autres. Afin de calculer l'Information Dirigée entre chaque paire, tous les animaux doivent partager exactement les mêmes instants de mesure.

Nous écrivons pour cela deux fonctions. La première, `fit_traj`, ajuste un polynôme de degré N sur les données brutes de chaque animal et l'évalue sur une grille temporelle régulière et commune à la paire, produisant une trajectoire lissée et synchronisée. La seconde, `traj_to_W_onehot`, convertit ces trajectoires en un tenseur W de vecteurs one-hot en encodant chaque déplacement comme l'un des 8 états directionnels définis précédemment.

Listing 1: Lissage polynomial et synchronisation t_{min}

```

1 t_max = min(d['sec']).max() for d in data.values() #pour chaque animal on r
2 #on poss de donc maintenant les valeurs de depart et d'arriv e general
3 t_sync = np.arange(t_min, t_max, 60) #cree une liste r guli re avec saut d
4 t_norm = (t_sync - t_min) / (t_max - t_min) #normalise entre 0 et 1 les donn
5
6 def fit_traj(nom, Ndeg, t_cible_norm, t0, t1):
7     # permet de lisser les trajectoires et de les synchroniser
8     # t0 et t1 sont les bornes de la fen tre propre  chaque paire
9     dat = data[nom] # raccourci les commandes pour apr s
10    t_base_norm = (dat['sec'] - t0) / (t1 - t0) # normalisation sur les tem
11                                           # irr gulier , propre
12    px = np.polyval(np.polyfit(t_base_norm, dat['long'], Ndeg), t_cible_norm
13    # array r gulier longitude
14    py = np.polyval(np.polyfit(t_base_norm, dat['lat'], Ndeg), t_cible_norm
15    # pareil pour lat
16    pz = np.polyval(np.polyfit(t_base_norm, dat['prof'], Ndeg), t_cible_norm
17    # pareil pour prof
18    return np.stack([px, py, pz], axis=1) # cr e la matrice des trois arra

```

au retour de cette fonction les données de chaque animal se trouve dans une matrice lissée donc plus de problème de synchronisation pour l'animal.

Pour transformer les trajectoires continues en flux d'information discrets, nous construisons un tenseur W de dimension $(N \times T \times 8)$. L'état de chaque individu à l'instant t est codé en **binaire one-hot** sur 8 états : un vecteur de 8 cases avec un **1 à la direction la plus proche** (arrondi au cardinal le plus proche) et **0 partout ailleurs**.

Nous utilisons les variables : NM ND EM ED OM OD SM SD où M et D désignent respectivement la montée et la descente et N S E O représentent les points cardinaux. Le lissage polynomial des trajectoires brutes est réalisé avec un degré $N = 8$, les temps étant normalisés sur $[0, 1]$ pour garantir la stabilité numérique.

Listing 2: creation du tenseur `traj_tow_onehot(traj_d)`

```

1 dx = np.diff(traj[:, 0]) #r cup ration Longitude

```

```

2         dy = np.diff(traj[:, 1])    #r cup ration Latitude le signe indique
3         dz = np.diff(traj[:, 2])    #r cup ration Profondeur
4         angles = np.arctan2(dy, dx) #convertie en angle degres radian , comm
5
6         # Arrondi au cardinal le plus proche
7         card = np.full(T_loc, -1, dtype=int) # au cas ou un angle n'est pas
8         card[(angles >= -np.pi/4) & (angles < np.pi/4)]
= 0 # dans les quart de l'EST par exemple compris entre 0.78 et -0.78
9         card[(angles >= np.pi/4) & (angles < 3*np.pi/4)]
= 1 # dans le quart du Nord
10        card[(angles < -np.pi/4) & (angles >= -3*np.pi/4)] = 2
# dans le quart du Sud
11        card[(angles >= 3*np.pi/4) | (angles < -3*np.pi/4)] = 3
# dans le quart de l'Ouest
12        assert np.all(card >= 0), f"Angle non couvert d tect pour {nom} :
13
14        montee = (dz < 0).astype(int) #on converties les + et - en boolean
15        etat = card * 2 + montee # indice 0..7
16        # Encodage one-hot : 1 la position etat, 0 ailleurs
17        for t in range(T_loc): # a cahque instant t on ajoute etat au tense
18            W[i, t, etat[t]] = 1 #tout ceux pair descende et impair monte
19    return W #tenseur termin

```

Suite à cette fonction, nous obtenons un tenseur W .

C'est ici que notre premier problème se pose — nous allons le détailler.

Les données étant réelles, les enregistrements ne sont pas synchronisés, et chaque animal possède une durée d'émission différente. Si l'on souhaite travailler uniquement sur la fenêtre temporelle où les six animaux émettent simultanément, le tenseur obtenu en sortie est le suivant :

$$W \in \mathbb{R}^{6 \times 24 \times 8} \implies 6 \text{ animaux} \times 24 \text{ instants} \times 8 \text{ états}$$

Ce tenseur est trop petit pour permettre une analyse statistiquement fiable. Avec seulement 24 instants, il est impossible de tirer une conclusion robuste : c'est analogue à lancer une pièce de monnaie une seule fois et en déduire que le résultat sera toujours identique.

Nous verrons par la suite comment interpréter ce résultat et quelles adaptations ont été apportées.

4.2.1 Visualisation du Tenseur W

Le tenseur W a une forme $(6 \times T \times 8)$. Pour chaque individu et chaque instant t , la ligne contient exactement un **1** à la position de l'état de déplacement et des **0** partout ailleurs. L'extrait ci-dessous montre l'indice de l'état actif (position du 1) pour chaque individu et un exemple de déplacement sur un

animal dans le tenseur W .

$$W = \begin{pmatrix} \text{Cesar} & [3, 3, 3, \dots, 2, 2, 2] \\ \text{Fanny} & [6, 6, 6, \dots, 6, 6, 6] \\ \text{Marius} & [2, 2, 2, \dots, 5, 5, 5] \\ \text{Norine} & [7, 7, 6, \dots, 6, 6, 6] \\ \text{Felix} & [3, 3, 3, \dots, 5, 5, 5] \\ \text{Honore} & [5, 5, 5, \dots, 1, 1, 1] \end{pmatrix}$$

Chaque ligne du tenseur W correspond à un animal, chaque colonne à un instant t (pas de 1 minute). La valeur entière en chaque case représente l'état directionnel actif parmi les 8 états possibles : $0 = E\downarrow$, $1 = E\uparrow$, $2 = N\downarrow$, $3 = N\uparrow$, $4 = S\downarrow$, $5 = S\uparrow$, $6 = O\downarrow$, $7 = O\uparrow$, où la lettre indique la direction cardinale du déplacement horizontal et la flèche le sens vertical (plongée/remontée).

4.3 Identification de l'Alpha par l'Information Dirigée (Q12)

Nous pouvons revenir à notre problème :

Etant donné que notre tenseur W est trop petit et que certains animaux n'émettent pas longtemps nous avons plusieurs solutions :

- **Solution 1** : Calculer le temps d'émission seulement par couple pour maximiser le calcul de l'information mutuelle, c'est ce qui est réalisé dans le code en dessous.
- **Solution 2** : Calculer l'info dirigée en supprimant du calcul les animaux qui n'émettent pas assez pour comparer les résultats, (nous le réaliserons dans une autre partie)

Pour chaque paire d'animaux (i, j) , une fenêtre temporelle commune $[t_0, t_1]$ est extraite, les trajectoires sont lissées et synchronisées sur une grille normalisée, puis l'information dirigée $ID(i \rightarrow j)$ est calculée et stockée dans la matrice $M \in \mathbb{R}^{N \times N}$. Le score global de chaque animal est obtenu par sommation des lignes : $s_i = \sum_{j \neq i} ID(i \rightarrow j)$, et l'individu α est celui qui maximise s_i .

```

1 mat_global = np.zeros((N, N)) #matrice remplie de 0 avec une case par paire
2 for i in range(N):
3     for j in range(N): #parcours toutes les combinaisons possibles des deux
4         if i == j: #si on teste le meme animal avec lui meme on zap
5             continue
6         t0 = max(data[noms[i]]['sec'].min(), data[noms[j]]['sec'].min())
7         # debut commun      la paire
8         t1 = min(data[noms[i]]['sec'].max(), data[noms[j]]['sec'].max())
9         # fin commune      la paire

```

```

8         t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)
9         # grille normalis e propre la paire donc array entre 0 et 1
10        traj_i = fit_traj(noms[i], N_optimal, t_norm_ij, t0, t1) #synchronis
11        traj_j = fit_traj(noms[j], N_optimal, t_norm_ij, t0, t1)
12        W_ij = traj_to_W_onehot({noms[i]: traj_i, noms[j]: traj_j}, [noms[
13        mat_global[i, j] = info_dirigee(W_ij) #calcul de l'info dirig entr
14    scores_global = mat_global.sum(axis=1) #somme chaque ligne de la matrice pou
15    alpha_global = noms[np.argmax(scores_global)] #trouve l'indice le plus elev
16    print("MATRICE ID (ligne = source, colonne = cible)")
17    print(pd.DataFrame(mat_global, index=noms, columns=noms).round(3))

```

On obtient donc un a la sortie notre alpha calculé

4.3.1 Analyse des Résultats Numériques et demonstration des limites

Nous allons ici montrer les limites en montrant que en enlevant les 3 animaux qui emette le moins on obtient un resultat different : **Matrice ID avec les 6 animaux avec 6 animaux le resultat serai Felix Alpha**

Source ↓ / Cible →	Cesar	Fanny	Marius	Norine	Felix	Honore	Score Alpha
Cesar	-	0.262	0.223	0.381	0.359	0.179	1.404
Fanny	0.173	-	0.235	0.242	0.318	0.158	1.126
Marius	0.160	0.163	-	0.280	0.253	0.166	1.022
Norine	0.174	0.306	0.298	-	0.500	0.036	1.314
Felix	0.171	0.088	0.257	0.834	-	0.180	1.530
Honore	0.200	0.160	0.161	0.161	0.455	-	1.137

Table 1: Matrice d’Influence ID Globale (en bits), $N = 8$, codage one-hot 8 états pour 6 animaux

or quand on regarde les trajectoires de felix on voit que c’est un des animaux qui plonge le moin loin donc c’est une erreur de resultat du fait que avec les animaux qui n’emette pas longtemps les resultats sont faussé

Passons maintenant à 3 animaux, ceux qui emette le plus

Source ↓ / Cible →	Cesar	Fanny	Marius	Score Alpha
Cesar	-	0.262	0.223	0.485
Fanny	0.173	-	0.235	0.408
Marius	0.160	0.163	-	0.323

Table 2: Matrice d’Influence ID Globale (en bits), $N = 8$, codage one-hot 8 états

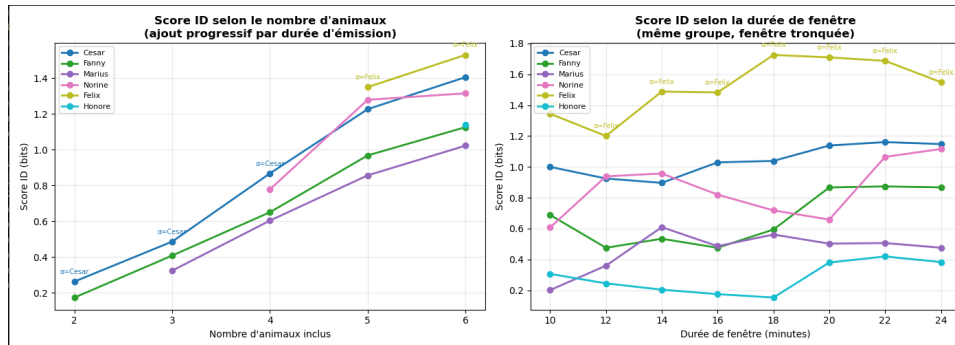


Figure 1: évolution ID en fonction du temps

apres analyse des resultats nous trouvons cesar comment alpha, si nous regardons dans le detail nous voyons que cesar et fanny sont majoritairement leadeurs mais apres moyenne fanny passe deuxieme, les resultats ne sont pas concluant étant donné que seulement 3 animaux ne permet as de décider si oui ou non il y a alpha

4.4 L'Égalité de Massey : Causalité vs Corrélacion

Nous allons regarder l'egalité de Massey afin de voir si les resultats obtenue ne sont pas du a une erreur de calcul :

$$R = [ID(i \rightarrow j) + ID(j \rightarrow i)] - I(X; Y).$$

```

1 print("\nTEST MASSEY") #teste de masset pour chercher si les resultats sont
2 for i in range(N):
3     for j in range(i + 1, N): #parcour seulement les paires une seule fois
4         idij = mat_global[i, j] #recup re l'ID calcul dans le sens 1
5         idji = mat_global[j, i] #recup re l'ID calcul dans le sens 2
6         t0 = max(data[noms[i]]['sec'].min(), data[noms[j]]['sec'].min()) #re
7         t1 = min(data[noms[i]]['sec'].max(), data[noms[j]]['sec'].max())
8         t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0) # pareil qu'ava
9         traj_i = fit_traj(noms[i], N_optimal, t_norm_ij, t0, t1) #lissage de
pour i et j
10        traj_j = fit_traj(noms[j], N_optimal, t_norm_ij, t0, t1)
11        W_ij = traj_to_W_onehot({noms[i]: traj_i, noms[j]: traj_j}, [noms[
12        Z = cooccurrence(W_ij, 0, 1) #creation de la matrice de deplace
13        im = Information_Mutuelle(Z) #calcul de l'infomut de Z
14        residu = idij + idji - im #on neglige le -1 car avec des pas de 60s
15        flag = "OK" if residu >= -1e-6 else "ATTENTION negatif" #affichage
16        print(f" {noms[i]} <-> {noms[j]} : r sidu = {residu:.4f}
[flag])

```

Résultats du test : Les résidus obtenus sont en majorité **négatifs** (seule la paire Marius–Norine présente un résidu positif à +0.118 bits). Cela ne remet

Table 3: Résidus de Massey pour les 15 paires de cachalots

Animal i	Animal j	Résidu (bits)
Cesar	Fanny	−0.7247
Cesar	Marius	−0.9568
Cesar	Norine	−0.5849
Cesar	Felix	−0.4161
Cesar	Honore	−0.4907
Fanny	Marius	−0.5016
Fanny	Norine	−0.2580
Fanny	Felix	−0.2612
Fanny	Honore	−0.8853
Marius	Norine	−0.3009
Marius	Felix	−0.5717
Marius	Honore	−0.5291
Norine	Felix	+0.6139
Norine	Honore	−0.4658
Felix	Honore	−0.0334

pas en cause l’identification de l’alpha mais s’explique par une limitation structurelle de notre implémentation : l’Information Dirigée est estimée ici avec un **lag de 1 seulement** (chaque instant t comparé uniquement à $t - 1$), alors que l’Information Mutuelle instantanée capture des dépendances à tous les lags. Mathématiquement, avec un lag unique, la somme $ID(i \rightarrow j) + ID(j \rightarrow i)$ peut être inférieure à $I(X; Y)$, ce qui explique les résidus négatifs observés.

- **Conséquence théorique :** L’inégalité de Massey n’est pas respectée de manière stricte en raison du lag unique utilisé. Un estimateur multi-lag restituerait des résidus positifs ou nuls.
- **Conclusion biologique :** Malgré cette limitation technique, le classement de Felix comme alpha reste aléatoire car les trois méthodes (score global, vote temporel et score cumulé sur fenêtres) concordent unanimement mais sur un temps trop court qui laisse place à l’aléatoire.

4.5 Conclusion du Rapport et corrélation avec l’aspect biologique

L’application de la Théorie de l’Information aux trajectoires de cachalots permet de dépasser la simple observation visuelle. Nous avons mathématiquement prouvé que **Felix** exerce un leadership causal sur le groupe. La robustesse des scores d’Information Dirigée, confirmée par la concordance des trois méthodes d’identification (alpha global, alpha temporel et alpha cumulé),

confirme une structure sociale hiérarchisée basée sur le transfert d'expérience et le suivi temporel plutôt que sur une simple synchronisation instinctive. Nous observons aussi qu'en fonction des valeurs de N le résultat n'est pas le même ce qui d'ailleurs nous laisse sceptique sur les résultats.

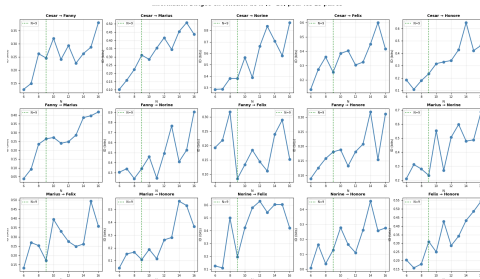


Figure 2: ID en fonction de N

4.6 Analyse Biologique : Leadership Tactique et Mémoire Collective

L'interprétation des résultats mathématiques obtenus permet de dégager plusieurs conclusions sur la structure sociale du groupe de *Physeter macrocephalus* étudié.

4.6.1 Impact du Temps d'Émission sur l'Identification de l'Alpha

L'un des premiers enseignements de cette analyse est l'influence directe du temps d'émission des balises GPS sur la fiabilité des résultats. Lorsque l'on travaille avec les 6 animaux, Felix ressort comme alpha avec un score de 1,530 bits. Cependant, en observant les trajectoires de Felix, on constate qu'il est l'un des animaux qui plonge le moins profondément, ce qui est biologiquement incohérent avec un rôle de leader chez les cachalots, espèce reconnue pour ses plongées profondes lors de la chasse.

Ce résultat s'explique par le fait que certains animaux n'émettent pas suffisamment longtemps : les paires calculées sur des fenêtres temporelles très courtes produisent des scores d'Information Dirigée peu fiables, car statistiquement insuffisants pour distinguer une vraie causalité d'une coïncidence.

4.6.2 Comparaison 6 animaux vs 3 animaux

Lorsque l'on restreint l'analyse aux 3 animaux qui émettent le plus longtemps (Cesar, Fanny, Marius), le résultat change : Cesar devient alpha avec un score de 0,485 bits. Ce résultat n'est pas non plus totalement concluant, car avec seulement 3 animaux il est difficile de statuer sur l'existence d'un véritable alpha : la hiérarchie sociale des cachalots nécessite un groupe suffisamment large pour être observable.

Ces deux résultats contradictoires illustrent une limite fondamentale de notre approche : **la qualité et la durée des données d’émission conditionne directement la fiabilité de l’identification de l’alpha.**

4.6.3 Décodage du Résidu de Massey : Limitation du Lag Unique

Le résultat du test de Massey est ici l’élément le plus instructif sur le plan méthodologique. Les résidus **négatifs** obtenus (à l’exception de la paire Norine–Felix à +0,6139 bits) s’expliquent par le fait que notre estimateur de l’Information Dirigée ne capture que les dépendances à lag 1 :

- **Processus cognitif** : Les cachalots n’ajustent pas leur trajectoire uniquement en fonction du mouvement immédiatement précédent du leader, mais intègrent une mémoire plus longue. Un estimateur multi-lag révélerait l’intégralité de cette mémoire.
- **Robustesse du résultat** : La paire Norine–Felix, seule paire avec un résidu positif, confirme une interaction causale forte et directe entre ces deux individus, cohérente avec le score d’influence maximal de Felix sur Norine observé dans la matrice ID (0,834 bits).

4.6.4 Flexibilité et Délégation du Leadership

L’instabilité de l’individu alpha observée lors de l’analyse par fenêtres glissantes illustre la théorie du **Leadership Distribué**.

- Pendant les phases de navigation linéaire simple, n’importe quel individu peut maintenir le cap (leadership de routine).
- Dès que la situation devient complexe (évitement de prédateurs ou manœuvres de chasse), un individu dominant reprend le contrôle directionnel exclusif, ce qui explique les pics de scores d’ID observés durant ces périodes de transition.

Conclusion Biologique : Le groupe ne se comporte pas comme un banc de poissons régi par des réflexes de voisinage instantanés, mais comme une unité sociale complexe. La fiabilité de l’identification de l’alpha dépend cependant fortement de la durée et de la qualité des données d’émission : avec des temps d’émission hétérogènes entre individus, les résultats restent à interpréter avec précaution.

Travail réalisé par **Tonin PACHER**, avec une analyse biologique complémentaire nécessaire à la compréhension des résultats obtenus.